

Домашнее задание 3

Лиза Фоменко

Задание 1

К серверу приходят одновременно n клиентов. Для клиента i известно время его обслуживания t_i . Время ожидания клиента определяется как сумма времени обслуживания всех предыдущих клиентов и времени обслуживания его самого. К примеру, если обслуживает клиентов в порядке номеров, то время ожидания клиента i будет равно $\sum_{j=1}^i t_j$. Постройте эффективный алгоритм, находящий последовательность обслуживания клиентов с минимальным суммарным временем ожидания клиентов, доказжите его корректность и оцените асимптотику.

на клиента номер j : $\sum_{j=1}^i t_j$

все клиенты: $\sum_{i=1}^n \sum_{j=1}^i t_j$

Нам необходимо минимизировать функцию всех клиентов. Давайте распишем ее подробнее:

$$\sum_{i=1}^n \sum_{j=1}^i t_j = \sum_{j=1}^1 t_j + \sum_{j=1}^2 t_j + \dots + \sum_{j=1}^{n-1} t_j + \sum_{j=1}^n t_j = t_1 + (t_1 + t_2) + \dots + (t_1 + t_2 + \dots + t_{n-1}) + (t_1 + t_2 + \dots + t_{n-1} + t_n) = n * t_1 + (n-1) * t_2 + \dots + 2 * t_{n-1} + t_n$$

Очевидно, что для минимизации данной функции нам необходимо так расположить клиентов, чтобы $t_1 \leq t_2 \leq \dots \leq t_{n-1} \leq t_n$.

Докажем очевидное: пусть, без ограничения общности, существует клиент i и клиент j , при чем $t_i < t_j$, но в оптимальном порядке наших клиентов t_j будет стоять раньше. Рассмотрим две очереди: в одной i стоит раньше j , в другой – наоборот (на тех же местах, но наоборот, с инверсией). Тогда распишем сумму времени клиентов:

$$\text{queue1} = n * t_1 + (n-1) * t_2 + \dots + (n-i+1) * t_i + \dots + (n-j+1) * t_j + 2 * t_{n-1} + t_n = \text{CommonPart} + (n-i+1) * t_i + (n-j+1) * t_j - \text{по предположению больше (так как уже не оптимальна)}$$

$$\text{queue2} = n * t_1 + (n-1) * t_2 + \dots + (n-i+1) * t_j + \dots + (n-j+1) * t_i + 2 * t_{n-1} + t_n = \text{CommonPart} + (n-i+1) * t_j + (n-j+1) * t_i - \text{по предположению оптимальна}$$

Вычтем из queue2 queue1 (ожидаем результат < 0 , так как в оптимальной очереди сумма времен минимальна):

$$\text{queue2} - \text{queue1} = \text{CommonPart} + (n-i+1) * t_j + (n-j+1) * t_i - \text{CommonPart} - (n-i+1) * t_i - (n-j+1) * t_j = (n-i+1) * t_j + (n-j+1) * t_i - (n-i+1) * t_i - (n-j+1) * t_j = (n-i+1) * (t_j - t_i) + (n-j+1) * (t_i - t_j) = (t_j - t_i) * (n-i+1 - n+j-1) = (t_j - t_i) * (j-i)$$

По предположению: $t_i < t_j; i < j \Rightarrow (t_j - t_i) * (j-i) > 0 \Rightarrow \text{queue2 не оптимальна} \Rightarrow \text{наше предположение неверно} \Rightarrow \text{клиентов необходимо расположить в порядке увеличения времен обслуживания.}$

Дело за малым - осталось отсортировать массив времен обслуживания клиентов. Вариантов сортировок массива множество. Нижняя граница сортировок сравнением (в худшем случае среди всех алгоритмов) – $\Omega(n \log n)$; я бы сделала MergeSort и осталась довольна за $\Theta(n \log n)$ (тут мы не можем использовать сортировку подсчетом, так как не время ожидания $\in \mathbb{R}$)

Задание 2

На вход поступает n котов целочисленной массы от 2 до k килограммов. Для каждого кота известна масса и кличка. Известно, что сначала накормить требуется наиболее худосочных. Предложите алгоритм, выводящий порядок, в котором нужно кормить котов, доказжите его корректность и оцените асимптотику.

Условие отлично подходит для сортировки подсчетом (потому что а) у нас целые числа; б) дан возможный диапазон весов от 2 до k). Правда в случае, когда $k \gg n$, мы потратим достаточно памяти, но, прибегнув к здравому смыслу и биологическому образованию, поймем, что $k \leq 25$. В общем, решаем сортировкой подсчетом, которая имеет асимптотику $\Theta(n)$.

Не знаю, зачем в задаче говорится о кличке котов, потому что она ни на что не влияет, и по условию все коты одинаковой массы одинаковы (мы не можем их различить).

Сортировка подсчетом основана на нахождении количества элементов, которые не превосходят каждое из возможных чисел (такая отсылка к куммулятивной функции распределения).

Алгоритм сортировки подсчетом:

```
arr # исходный массив длины n
sorted_arr # отсортированный выход алгоритма длины n
C = [0, ... 0] # длины k-2

for i in range(n): # по всему массиву -- ровно n шагов
    C[A[i]] += 1 # [A[i]] - элемент массива от 2 до k. C[A[i]] - счетчик элементов, равных [A[i]]

# сейчас C = [n2, n3, ... n_k], где n_i - количество элементов, равных i
# теперь необходимо для каждого i = 2, ..., k найдем количество элементов, не превосходящих i

for j in range(1, k-2):
    C[i] = C[i] + C[i-1]
```

сейчас $C = [n_2, n_3, \dots, n_k]$, где n_i - количество элементов, не превосходящих ($\leq$) i

Рассмотрим $C[i] = m$ и $C[i+1] = h$. Мы знаем, что количество элементов $\leq i$ равно m , а количество элементов $\leq i+1$ равно h . Отсюда мы выводим, что у нас ровно $m-h$ элементов равных $i+1$, при этом стоят они на позициях от $m+1$ до h . Осталось использовать это тайное знание для расставления элементов из массива arr в массив $sorted_{arr}$, где на позициях с $m+1$ до h будет стоять число $i+1$.

Вариантов как это сделать мб несколько, я люблю вариант из Кормена, так как в такой имплементации алгоритм остается устойчивым к порядку одинаковых элементов (то есть если в arr мурка 3 кг была раньше жоржа 3 кг, то в $sorted_{arr}$ мурка также будет стоять до жоржа).

```
for j in range(n, -1):
    B[C[A[j]]] = A[j]
    C[A[j]] -= 1
```

Оценим ассимптотику: в первом цикле выполняется ровно n шагов, каждый шаг совершается операция сложения чисел за $\Theta(1) \Rightarrow$ данный цикл сработает за $\Theta(n)$.

Во втором цикле выполняется $k - 2$ шагов, каждый шаг совершается операция сложения чисел за $\Theta(1) \Rightarrow$ данный цикл сработает за $\Theta(k - 2) = \Theta(k)$.

Последний цикл также делает n шагов по два действия за $\Theta(1)$ (присоединение элемента массива и мат операция), поэтому данный цикл сработает за $\Theta(n)$.

Общий алгоритм работает за $\Theta(n) + \Theta(k - 2) + \Theta(n) = \Theta(2n + k - 2) = \Theta(n)$ (обычно $n \gg k$).

Задание 3

На вход задачи поступают три отсортированных массива. Постройте алгоритм, находящий число уникальных элементов в объединении этих массивов.

В задаче ничего не говорится о самих массивах: состоят они из уникальных элементов или нет? Так как надеяться на уникальность элементов мы не можем, то решаем более хардкорную задачу (то есть в массиве могут быть повторения $\Rightarrow$ стоят подряд!) в стиле mergesort.

```
arr1, arr2, arr3 # наши массивы
point1 = point2 = point3 = 0 # инициализируем указатели
cnt = 0 # счетчик
n1, n2, n3 = len(arr1), len(arr2), len(arr3) # длины массивов. будем их считать за O(1) как в питоне

while p1 < n1 and p2 < n2 and p3 < n3: # пока все указатели можно увеличивать

    # сначала ищем минимум из элементов на соотв указателях. так как нам нужно еще понять,
    # из какого именно массива наименьший элемент, распишем все if ))

    el1, el2, el3 = arr1[p1], arr2[p2], arr3[p3]

    if el1 > el2:
        if el2 > el3: # p3 минимум из новых элементов
            min = el3
            p3 += 1
```

```

        while arr3[p3] == min: # убираем одинаковые элементы подряд в arr3
            p3 += 1
        result += 1

    elif el2 < el3:
        min = el2
        p2 += 1
        while arr2[p2] == min: # убираем одинаковые элементы подряд в arr2
            p2 += 1
        result += 1

    else:
        # el2 == el3 => нужно только один добавить
        min = el2
        while arr2[p2] == min: # убираем одинаковые элементы подряд в arr2
            p2 += 1
        while arr3[p3] == min: # убираем одинаковые элементы подряд в arr3
            p3 += 1
        result += 1

# аналогичные блоки когда для
elif el1 < el2:
    .....

else: # el1 == el2
    .....

# здесь один из массивов закончился => ищем уникальных элементы в оставшихся двух массивах, п
# пока не останется один массив
while p1 < n1 and p2 < n2 :    # arr3 закончился

    # алгоритм точно такой же! но тут уже 2 массива как в мерджсорте
    el1, el2 = arr1[p1], arr2[p2]

    if el1 > el2:
        min = el2
        while arr2[p2] == min: # убираем одинаковые элементы подряд в arr2
            p2 += 1
        result += 1
    elif el1 < el2:
        min = el1
        while arr1[p1] == min: # убираем одинаковые элементы подряд в arr1
            p1 += 1
        result += 1

    else:
        # el2 == el1 => нужно только один добавить
        min = el1
        while arr1[p1] == min: # убираем одинаковые элементы подряд в arr1
            p1 += 1
        while arr2[p2] == min: # убираем одинаковые элементы подряд в arr2
            p2 += 1
        result += 1

# и когда у нас останется только один массив просто добавим не встречавшиеся в нем элементы :)

while p1 < n1:
    min = el1
    while arr1[p1] == min: # убираем одинаковые элементы подряд в arr1
        p1 += 1

```

```
result += 1
```

Данный псевдокод (который получился оч здоровый, но наглядный) описывает алгоритм решения нашей задачи. Он крайне похож на Merge из MergeSort-a. Идея аналогична MergeSort-y: так как наши массивы уже отсортированы, то минимальный элемент всех трех массивов - это минимальный из первых элементов массивов, поиск минимума займет всего $O(1)$.

Ищем минимума итеративно. Первый шаг: рассматриваем первый из ранее не рассмотренных элементов в каждом массиве и ищем минимум из них. считаем его (и избавляемся от всех его дубликатов). сдвигаем указатели в массивах, если в них встречается минимальный элемент шага.

Шаг n : ищем минимум из элементов каждого массива, на которых находятся указатели. сдвигаем указатели, избавляясь от этого минимума во всех массивах.

Тогда на каждом шаге мы делаем 3 сравнения (ищем минимальный из $el1, el2, el3$). В моем коде отражено, что если минимальных элементов несколько (например, $el2 = el3$), то мы увеличиваем счетчик обоих массивов (так как на следующей итерации цикла минимальным будет тот же по значению элемент; но на количество шагов это не влияет. просто удобно мне было).

Когда мы находим минимум, мы также избавляемся от его дубликатов во всех массивах (в случае, когда в массиве один элемент повторяется, мы не считаем его несколько раз). Это можно делать потому, что я на каждом шаге убираю сразу все элементы, которые равны current minimum $\Rightarrow$ мы никогда не встретим этот элемент позже, а по построению алгоритма не встречали его раньше (массивы отсортированы).

Чтобы рассмотреть оценку алгоритма, мы сначала обратимся к случаю 2 массивов (как в mergesort). На каждом шаге мы берем по элементу из каждого массива (за $O(1)$) и сравниваем элементы: рассматриваем три случая ($>$, $<$, $=$) - данная операция выполняется за $O(1)$. Сколько таких сравнений мы можем сделать? от $\min(n1, n2)$ до $\max(n1, n2)$. Затем мы добавляем оставшиеся элементы из массивов, всего сделается $n1+n2$ сравнений. Замечу, что когда мы сравниваем элементы одного массива друг с другом в цикле while (когда нашли current minimum и хотим убрать все эти элементы), то мы не можем сделать больше n_k сравнений $\Rightarrow$ общее количество сравнений тоже $m+m \Rightarrow$ ассимптотика $\Theta(n1 + n2) = \Theta(\max(n1, n2))$.

В случае трех массивов логика остается такой же. Изначально мы сравниваем 3 элемента за $O(1)$, хотя делаем уже 2 сравнения (например, $el1$ с $el2$ и $el2$ с $el3$ - транзитивность работает на нас; при этом на каждую пару есть 3 возможных исхода - $>$, $<$, $=$). После этого мы убираем дубликаты из массивов, равные найденному минимуму. Оценим ассимптотику с помощью указателей: на каждый сдвиг одного указателя выполняется $\Theta(1)$ действий (сравнение/я). $p1$ сдвинется ровно $n1$ раз, $p2$ - $n2$, $p3$ - $n3$, откуда общее число сдвигов составляет $n1+n2+n3$, каждый шаг занимает $\Theta(1) \Rightarrow$ сложность алгоритма $\Theta(n1 + n2 + n3) = \Theta(\max(n1, n2, n3))$

Задание 4

На вход задачи поступает массив a из n чисел. Постройте алгоритм, находящий число инверсий в массиве, то есть таких пар индексов i, j , что $i < j$ и $a[i] > a[j]$.

Для решения этой задачи почти в чистом виде используют MergeSort. Заметим, что после разбивки массива на подмассивы длины 1, мы получим n одноэлементных массивов, которые для merge мы будем разбивать на пары, обычно по принципу 1-2, 3-4,

Как понять, что на уровне листьев дерева MergeSort произошла инверсия? В отсортированном массиве в каждой паре элемент в первом массиве (при разбиении вида 1-2, 3-4...) будет меньше, чем элемент второго массива, поэтому в процедуре Merge элемент из первого массива пойдет до элемента массива второго.

Если же сначала мы добавим элемент из второго массива, значит, что элемент $arr_{i+1} < arr_i$ - определение инверсии.

Перейдя на следующий уровень Merge, мы получим два массива вида $[arr_i, arr_{i+1}], [arr_{i+2}, arr_{i+3}]$. В отсортированном массиве снова все элементы первого подмассива меньше элементов второго подмассива, поэтому при процедуре Merge мы можем посчитать число элементов второго подмассива, которые будут добавлены раньше элементов первого подмассива. Важно заметить, что на этом уровне мы не можем судить об инверсиях на младших уровнях, так как оба подмассива уже отсортированы. Но общее число инверсий будет подчиняться формуле $inv_{left} + inv_{right} + curr_{inv}$.

Для этого при рекурсивном вызове Merge, мы суммируем число инверсий первого подмассива процедуры и правого подмассива, а также подсчитываем число элементов, добавленных из второго подмассива раньше первого при Merge.

```
arr      # исходный массив
start = 0
end = n
```

```

Merge_inv_cnt(left, right):
    cnt = 0
    while p1 < n and p2 < n:
        if left[p1] > right[p2]:
            cnt += 1
            p2 += 1
        else:
            p1 += 1
    return cnt

inv_cnt(arr, start, end):
    left = arr[start, (start+end)//2]
    right = arr[(start+end)//2, end]
    inv_left = inv_cnt(arr, start, (start+end)//2)
    inv_right = inv_cnt(arr, (start+end)//2, end)
    curr_inv = Merge_inv_cnt(left, right)
    return inv_left + curr_inv + inv_right

```

На счет ассимптотики: мы добавляем только увеличение счетчика на некоторых шагах алгоритма, поэтому его ассимптотика не изменится и останется $\Theta(n \log n)$

Задание 5

На вход поступает число n и массив a размера $2n+1$. Постройте алгоритм, находящий число s , минимизирующее сумму $\sum_{i=1}^{2n+1} |a_i - s|$

Функция для минимизации очень похожа на МАЕ, только собственно без mean :)

Я знаю, что минимизация $MAE = \frac{1}{N} \sum_{i=1}^n |y_i - C|$ эквивалентна поиску медианы y , то есть минимизация МАЕ достигается при $C = y.median$. Докажу это нестрого: найдем производную

$$\sum_{i=1}^n |y_i - C|'_x = \sum_{i=1}^n \text{sign}(y_i - C) - \dots > 0$$

Необходимо найти такое значение C , чтобы сумма sing-ов была равна 0, то есть $n/2 - n/2 = 0$, то есть половина $y_i - C$ отрицательна, а половина - положительна. Так как C является элементом массива, то $C = y.meadian$ (можно и более строго доказать, но думаю так подойдет)

В нашем случае все аналогично: минимум $\sum_{i=1}^{2n+1} |a_i - s|$ достигается при $s = median(a)$, а это $n+1$ по счету элемент в отсортированном массиве.

Как эффективно найти медиану массива? Сортировать и брать середину, тратя на это $O(n \log n)$ или делать сортировку подсчетом за $O(n)$ скучно. Воспользуемся интересным свойством QuickSort.

Смысл QuickSort в том, что, работая с элементом `element`, мы ставим его на какое-либо место в массиве (in-place) так, что все числа до `element` не превосходят его, а все числа после – не меньше `element`, а это значит, что в процессе дальнейших действий `element` не будет изменять свою позицию в массиве. То есть поставив `element` в k -л ячейку массива, например k , мы знаем, что в отсортированном массиве `element` будет на k -ом месте, то есть является k -ым по возрастанию (если все элементы разны; если есть повторы, то первый элемент равный `element` может быть раньше в массиве).

алгоритм:

```

array = [a1,...a_{2n+1}]
start = 1, end = 2n+1
while k != n+1
    select pivot          # random
    k = QuickSort+(pivot, start, end)    # placed pivot into place k
    if k < n+1:
        start = k+1
    else:
        end = k-1

```

Делаем QuickSort+ до тех пор, пока не найдем элемент, который поставим на место $n+1$ – это и будет медиана. Однозначно определить ассимптотику нельзя. В среднем ассимптотика $O(N)$. Есть всякие эвристики QuickSort для избегания N^2 в худших случаях, но написанный мной в худшем случае даст именно квадратичную зависимость (например, если массив в обратном порядке отсортирован)

Задание 6

На вход подается массив $a_1, \dots, a_n$, в котором один из элементов встречается не меньше $\lceil \frac{n}{2} \rceil$ раз. Постройте алгоритм, находящий этот элемент.

Найти элемент можно, например, с помощью идеи сортировки подсчетом - считаем элементы по порядку массива. Если счетчик какого-либо из элементов достигает $n/2+1$, элемент найден, но такое решение займет кучу памяти (использование сортировки подсчетом обычно предполагает целые числа из ограниченного диапазона; у нас этих ограничений нет).

Будем использовать стратегию образования пар различных элементов: пусть у нас есть n элементов, при этом элемент $elem$ встречается $\lceil \frac{n}{2} \rceil$ раз. Это значит, что мы можем составить $\lfloor \frac{n}{2} \rfloor$ пар вида $(elem, a_i)$; где $elem$ - наш элемент, а a_i - другие элементы массива ($elem \neq a_i$). При этом останется хотя бы 1 $elem$ без пары (если $elem$ встречается БОЛЬШЕ $\lceil \frac{n}{2} \rceil$ раз).

Алгоритм работает за $O(N)$, так как N раз мы делаем шаг за $O(1)$: за шаг мы сохраняем наш элемент. На следующем шаге смотрим, совпадает ли новый элемент с предыдущим. Если элементы разные, двигаем указатель вперед и уменьшаем счетчик повторов на единицу. А вот если элементы совпали, то увеличиваем счетчик повторов элементов. Таким образом, мы делаем $N-1$ сравнение и на каждом шаге увеличиваем или уменьшаем счетчик на 1. Тогда в итоге мы получим либо значение больше нуля - есть искомый преобладающий элемент; значение равно 0 - преобладающих элементов нет.

Особая ситуация в случае, когда $elem$ составляет ровно половину элементов. Тогда наш элемент будет встречаться в каждой составленной паре. Я думаю, что можно сохранить эти пары и найти общий для них элемент.

Задание 7

Дан массив из n чисел. Нужно разбить этот массив на максимальное количество непрерывных подмассивов так, чтобы после сортировки элементов внутри каждого подмассива весь массив стал отсортированным. Предложите $O(n \log n)$ алгоритм для решения этой задачи.

Для решения этой задачи воспользуемся задачей 4. Там мы доказали, что для поиска количества инверсий в массиве мы используем mergesort, в котором при процедуре merge подсчитываем количество элементов "второго" массива (с большими границами), которые добавляются в результирующий массив раньше "первого" (который в исходном массиве стоит раньше).

Рассмотрим искомую задачу в случае, когда мы делим массив на два подмассива arr_{left} и arr_{right} . Пусть мы знаем, что отсортированные arr_{left} и arr_{right} составляют полностью отсортированный массив arr . Распишем формально:

$$arr_{left} \text{ sorted} \Rightarrow arr_{left_1} \leq arr_{left_2} \leq \dots \leq arr_{left_{k-1}} \leq arr_{left_k}$$

$$arr_{right} \text{ sorted} \Rightarrow arr_{left_{k+1}} \leq arr_{left_{k+2}} \leq \dots \leq arr_{left_{n-1}} \leq arr_{left_n}$$

Теперь, когда мы сделаем операцию merge (задание 4), то количество инверсий на данном шаге будет равно 0, так как все элементы левого подмассива меньше элементов правого подмассива (так как $arr_{left} + arr_{right}$ отсортирован, то есть $arr_1 \leq arr_2 \leq \dots \leq arr_{n-1} \leq arr_n$).

И в любом случае, когда мы получаем число инверсий $Merge_inv_cnt = 0$ на шаге merge, значит, что все элементы левого подмассива меньше элементов правого, следовательно, отсортировав каждый из них отдельно, мы получим отсортированный merged массив :)

Тогда нужно лишь немного модифицировать mergesort: так как мы ищем максимальное количество таких массивов, то нам подойдет классический mergesort, когда мы сливаем от наименьших к наибольшим массивам.

```
arr      # исходный массив
start = 0
end = n
result_borders = [] # здесь храним границы подмассивов
```

```
Merge_inv_cnt(left, right):
    cnt = 0
    while p1 < n and p2 < n:
        if left[p1] > right[p2]:
            cnt += 1
            p2 += 1
        return cnt
```

```
find_subarrays(arr, start, end):
```

```

left = arr[start, (start+end)//2]
right = arr[(start+end)//2, end]
curr_inv = Merge_inv_cnt(left, right)

#____вот тут находится изменение____#
if curr_inv == 0:
# значит все элементы left <= right => можем разбить arr[start, end] на left и right подмассивы

    result_borders.append([start, (start+end)//2], [(start+end)//2, end])
    # добавляем границы разбиения left и right

    # нам больше не нужно рассматривать этот участок.
    # нужно продолжить только с элементами слева и справа

    return 'found subarrays'
    (и продолжим поиск слева и справа от массива)
#_____#

return merge(left, right)

```

То есть когда мы найдем два подмассива, которые дают $curr_inv = 0$, то нам не нужно дальше мерджить получившийся общий подмассив, и мы продолжаем операции только слева от левой границы и справа от правой. Алгоритм работает за $\Theta(n \log n)$, так как работает как MergeSort (можно тоже доказать через дерево решений, но аналогично).